

THE NARRATIVE

Production Deployment Runbook · Railway + Vercel

Generated 2026-06-21 · target hosting ~\$30–\$50/mo

1 Overview & architecture

Two app services deploy from this repo to **Railway**: **api** (FastAPI under gunicorn) and **scheduler** (one process that runs every pipeline worker — scrape, embed, cluster, score, map, graph, exposure). Railway-managed **Postgres** (with pgvector) and **Redis** back them. The React/Vite frontend deploys to **Vercel** and proxies `/api` to the Railway API. The Consequence Propagation Engine is deterministic — **no LLM at serving time** — so running cost is dominated by hosting, not tokens.

Cost reality (measured): the entire Claude spend to build the current dataset (354 mapped events from 1,613 articles over 11 days) was **\$2.10** — about **\$6/month** at this ingestion rate. Hosting is the lever.

Lean	1 small VPS (docker-compose) + Vercel free	~\$15–\$25
Managed (chosen)	Railway 4 services + Vercel free	~\$30–\$50
Naive (avoid)	Deploy all 13 railway.toml services	~\$100–\$170

2 Pre-deploy checklist

- **Rotate** the Anthropic + Voyage keys in their consoles — the old ones leaked via the OneDrive `.env`. New keys go into Railway variables only, never committed.
- Confirm the canonical repo copy (three exist on disk; only the OneDrive copy is git-initialized).
- Generate a JWT secret: `openssl rand -hex 32`.

3 Push to GitHub

```
# from the project root (already git-initialized, 2 commits on main)
gh repo create the-narrative --private --source . --remote origin --push
# or manually:
git remote add origin https://github.com/<you>/the-narrative.git
git push -u origin main
```

4 Provision on Railway

- New Project → **Deploy from GitHub repo**. Railway reads `railway.toml` and creates the **api** + **scheduler** services.
- Add plugins: **New** → **Database** → **PostgreSQL**, and **New** → **Database** → **Redis**.
- Enable pgvector (the Alembic migration also runs `CREATE EXTENSION vector`):

```
psql "$DATABASE_URL" -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Set service Variables (both api & scheduler) — reference the plugin vars and add secrets:

```
DATABASE_URL = postgresql+asyncpg://... # from ${ Postgres.DATABASE_URL } (+asyncpg)
REDIS_URL    = ${ Redis.REDIS_URL }
APP_ENV      = production                # disables /docs, runs gunicorn
SECRET_KEY   = <openssl rand -hex 32>
ANTHROPIC_API_KEY = <rotated>
VOYAGE_API_KEY = <rotated>
ALLOWED_ORIGINS = https://<your-app>.vercel.app
APP_BASE_URL  = https://<your-app>.vercel.app
```

Full required/optional list is in `.env.production.example`. The api start command runs `alembic upgrade head` automatically.

5 Migrate the real data

The new Railway DB starts empty. A fresh dump of the real data (`scripts/narrative.dump`, 7.8 MB — 1,613 articles / 354 mapped) restores it so the site has data on day one. Regenerate then restore:

```
bash scripts/dump_for_railway.sh # refresh scripts/narrative.dump
pg_restore --no-owner --no-acl --clean --if-exists \
  -d "<RAILWAY_DATABASE_URL without +asyncpg>" scripts/narrative.dump
```

6 Deploy the frontend (Vercel)

- Import the GitHub repo in Vercel. Build is defined in `vercel.json` (`cd web && npm install && npm run build`).
- Set env var `VITE_MAPBOX_TOKEN` (Mapbox public token, free tier). **Do not** set `VITE_DEMO_MODE` — leaving it unset keeps the app on real data.
- Replace the `vercel.json` rewrite placeholder `REPLACE-WITH-RAILWAY-API...` with the real Railway API URL, commit, redeploy.
- Back on Railway, set `ALLOWED_ORIGINS` to the Vercel domain (CORS — keep it an explicit allowlist, never *).

7 Caps & cost controls

7.1 API rate / request caps

Per-user / per-IP throttling lives in `backend/api/rate_limit.py` and is Redis-backed (exact across gunicorn workers). Key = hashed bearer token (per-user, proxy-safe) with client-IP fallback. `/health` is exempt; the limiter fails open if Redis hiccups (never its own outage).

Global request cap	120 / minute	default_limits in rate_limit.py
Key	hashed token, else IP	rate_limit_key() in rate_limit.py
Store	Redis (REDIS_URL)	auto; memory:// if unset
Over-limit response	HTTP 429	slowapi handler in main.py

```
# tighten / loosen the global cap:
limiter = Limiter(key_func=rate_limit_key,
                  default_limits=["120/minute"], # <- edit here
                  storage_uri=storage_uri, swallow_errors=True)
```

7.2 Claude (token) cost caps

The only paid LLM step is `mapping_worker`, which is budget-routed (a small number of deep Claude calls per run). Cost alerting + budget are configured via env:

CLAUDE_DAILY_COST_ALERT_USD	20	alert when day's Claude spend exceeds it
CLAUDE_MONTHLY_BUDGET_USD	200	monthly budget ceiling for alerting
ADMIN_ALERT_EMAIL	(unset)	where budget alerts are emailed (needs SMTP)
CONSEQUENCE_ENGINE_MODEL	claude-sonnet-4	model used for mapping (cost driver)

Ingestion cadence also caps spend — slower intervals = fewer mappings = lower cost. Tune in env (`SCRAPE_INTERVAL_HOURS`, `MAPPING_INTERVAL_MINUTES`, thresholds `IMPORTANCE_THRESHOLD_DEEP/LIGHT`). Real measured rate: ~\$0.19/day. Live spend is visible in the `pipeline_metrics` table and the admin Cost dashboard.

To run with **zero** ongoing LLM spend, simply don't deploy the scheduler service — the api keeps serving the existing data and the deterministic exposure engine.

8 Post-deploy verification

```
curl https://<api>.up.railway.app/health # {"status":"ok","env":"production"}
# open the Vercel URL, sign in (enterprise@narrative.dev in non-prod), confirm:
#   - real events render (Feed / World / Deck / Exposure)
#   - /admin loads for an admin-tier user
#   - burst >120 req/min on an endpoint returns HTTP 429
#   - /docs is DISABLED (APP_ENV=production)
```

Verified locally before this runbook: 9/9 backend test files, 3/3 frontend test suites, production build, and all 11 main API endpoints returning real data.

9 Rollback & ops notes

- Railway keeps deploy history — roll back to a previous deployment from the service's Deployments tab.
- DB safety: snapshot before migrations with `pg_dump`; Railway Postgres also offers backups.

- Known follow-up: upgrade FastAPI/Starlette (0.111 predates Starlette DoS fixes) as a separate, tested change.
- Scheduler is optional; start it only when you want fresh ingestion (it also generates the event trajectory history shown in Event detail).

The Narrative — deployment runbook. Generated by build_deploy_pdf.py. Secrets are never included; fill real values only in the Railway/Vercel dashboards.